

Using logarithms to calculate probability in Binomial Distribution

By Jakub Grzana

Introduction

Binomial formula is very easy for humans to understand, but it requires a lot of computations so it's best to do it with a computer. However, there is a problem: formula uses newton's binomial, which itself uses factorials, and factorials are HUGE. Unless your programming language supports integers of arbitrary size, you need a trick to solve this problem. This trick is using logarithms.

Logarithmic representation of number

Let X be any real number other than 0 (in practice we don't need to exclude 0 in code, but let's keep mathematical approach here). In my notation, we can describe such number with two parameters: S and P , where S stands for *sign* and P stands for *power*.

$$\begin{aligned}X &= S * e^P \\ S &= \text{sign}(X) \\ P &= \ln(\text{abs}(X))\end{aligned}$$

I'm using following properties of logarithms: (*Note: base of logarithm isn't very important, as long as it's > 1. Natural logarithm was chosen arbitrarily*)

$$\begin{aligned}&\text{for } M, L > 0 \\ e^{\ln M} &= M \\ \uparrow M &\rightarrow \uparrow \ln(M) \\ \downarrow M &\rightarrow \downarrow \ln(M) \\ \ln(M * L) &= \ln(M) + \ln(L) \\ \ln\left(\frac{M}{L}\right) &= \ln(M) - \ln(L)\end{aligned}$$

I will explain shortly why is this useful. Now, let's define some basic operations on numbers in this notation.

$$\begin{aligned}X, Y &= S_i * e^{P_i} \\ X * Y &= (S_X \text{ xnor } S_Y) * e^{P_X + P_Y} \\ \frac{X}{Y} &= (S_X \text{ xnor } S_Y) * e^{P_X - P_Y} \\ \frac{X - Y}{X + Y} &\text{ can't be defined}\end{aligned}$$

We can define comparison, but it gets messy with signs. I didn't bother to find neat formulas for it.

$$\begin{aligned}X < Y &= P(X) < P(Y) \text{ for } X, Y \text{ of positive sign} \\ X < Y &= P(X) > P(Y) \text{ for } X, Y \text{ of negative sign} \\ X < Y &= \text{true for } S_X \text{ negative and } S_Y \text{ positive} \\ X < Y &= \text{false for } S_X \text{ positive and } S_Y \text{ negative}\end{aligned}$$

Why is this useful?

We can use it to store and perform some operations on very large numbers. Let's take factorial: we can define it for any integer N using following formula:

$$N! = \prod_{i=1}^N i$$

Factorial gets huge really fast.

$$\frac{(N+1)!}{N!} = N+1$$

Let's take some low numbers $N = 10$. Then $\frac{(N+2)!}{N!} = 11 * 12$. It means just by increasing N by 2, we increased factorial by (over) two orders of magnitude. And as N increases, factorial increases only faster!

N	N!	ln(N!)
1	1,00 E+00	0,00
2	2,00 E+00	0,69
3	6,00 E+00	1,79
4	2,40 E+01	3,18
5	1,20 E+02	4,79
6	7,20 E+02	6,58
7	5,04 E+03	8,53
8	4,03 E+04	10,60
9	3,63 E+05	12,80
10	3,63 E+06	15,10
11	3,99 E+07	17,50
12	4,79 E+08	19,99
13	6,23 E+09	22,55
14	8,72 E+10	25,19
15	1,31 E+12	27,90
16	2,09 E+13	30,67
17	3,56 E+14	33,51
18	6,40 E+15	36,40
19	1,22 E+17	39,34
20	2,43 E+18	42,34

Programming languages like C++ doesn't support integers of arbitrary precision, and even if they do it can be lost during division, when it must be converted into floating point (*Python has this problem*).

Binomial formula using logarithmic representation of number

Let's start with reminder about binomial formula. *Note that p here is different to P previously, we're not using logarithmic notation yet.*

$$P(X) = \binom{n}{X} p^X q^{n-X}$$
$$\binom{n}{X} = \frac{n!}{(n-X)! * X!}$$

p – probability of success in one trial
 $q = 1 - p$
 n – number of trials
 X – number of successes desired

Binomial formula can be used, for example, to determine “what are the odds of getting exactly X heads in n coin flips ($p=q=0.5$)”.

Let's consider boundaries:

$$\binom{n}{X} \geq 1$$
$$n \geq X \geq 0$$
$$p, q \in [0,1]$$
$$P(X) \in [0,1]$$

All numbers there are positive or zero, so I don't have to deal with signs and I will skip them entirely in logarithmic notation.

$$\ln(P(X)) = \ln\left(\binom{n}{X} p^X q^{n-X}\right) = \ln\left(\binom{n}{X}\right) + \ln(p^X) + \ln(q^{n-X})$$
$$\ln(p^X) = X * \ln(p)$$
$$\ln(q^{n-X}) = (n - X) * \ln(q)$$
$$\ln\left(\binom{n}{X}\right) = \ln\left(\frac{n!}{(n-X)! * X!}\right) = \ln(n!) - [\ln((n-X)!) + \ln(X!)]$$
$$\ln(n!) = \sum_{i=1}^n \ln(i)$$

Final result:

$$\ln(P(X)) = \sum_{i=1}^n \ln(i) - \sum_{i=1}^{n-X} \ln(i) - \sum_{i=1}^X \ln(i) + X * \ln(p) + (n - X) * \ln(q)$$
$$e^{\ln(P(X))} = P(X)$$

This formula look complicated only on the surface. It's very easy to implement. This approach allows us to perform all operations on logarithmic representations of numbers – because of this, we can compute $\binom{n}{X}$ which involves factorials of large integers.

Now you might have noticed we risk taking logarithm of $X = 0$ using this approach. In practice it isn't a problem, because computer will return lowest possible number in such case (“going to negative infinity”) which results in $e^{\ln(X)} \rightarrow 0$.

Conclusions

This approach allowed me to implement binomial formula in C++ with only “int” and “double” datatypes. It’s worth noting that logarithmic representation of number isn’t 100% accurate (*in the same way for computer $1/3 * 3 = 0.999999...$*) although with double-precision floating-point numbers it’s remarkably good.

As an example, I’ve taken values from Dream’s cheating scandal in Minecraft speedrunning. He got 42 enderpearls from 262 barbers. In bartering, there’s only 4,7% chance to get enderpearl, so to calculate odds of getting 42 or more, I’ve used binomial formula.

Result calculated by Matt Parker: $5,6 * 10^{-12}$

My result: $5,3 * 10^{-12}$

So the result is different, but it’s the same order of magnitude and difference is around 5%. Online calculators mostly returned just 0, so I think it’s acceptable result.